Pointers

Outline

- Concept of Pointer
- Pointers and its Applications
- Pointer Manipulation in C
- Pointers and Array
- Pointer Arrays
- Command Line Arguments
- Pointers to Functions

Concept of Pointer

- In memory, every stored data item occupies one or more contiguous memory cells.
  - The number of memory cells required to store a data item depends on its type (char, int, float, double, etc.).
- Whenever we declare a variable, the system allocates memory location(s) to hold the value of the variable.
  - Since every byte in memory has a unique address, this location will also have its own (unique) address.

Concept of Pointer

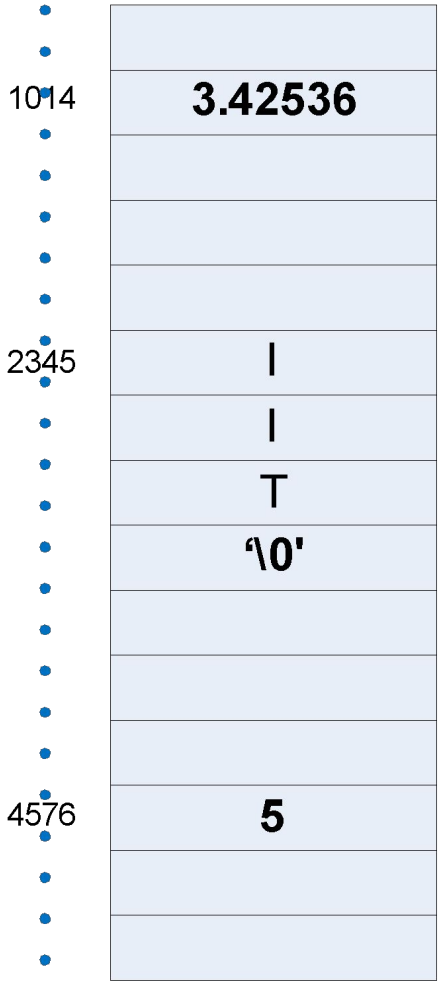
```
int y = 5;
char name[5]="IIT";
float x;
```

Memory locations

&y = 4576

&x = 1014

name[5] = 2345



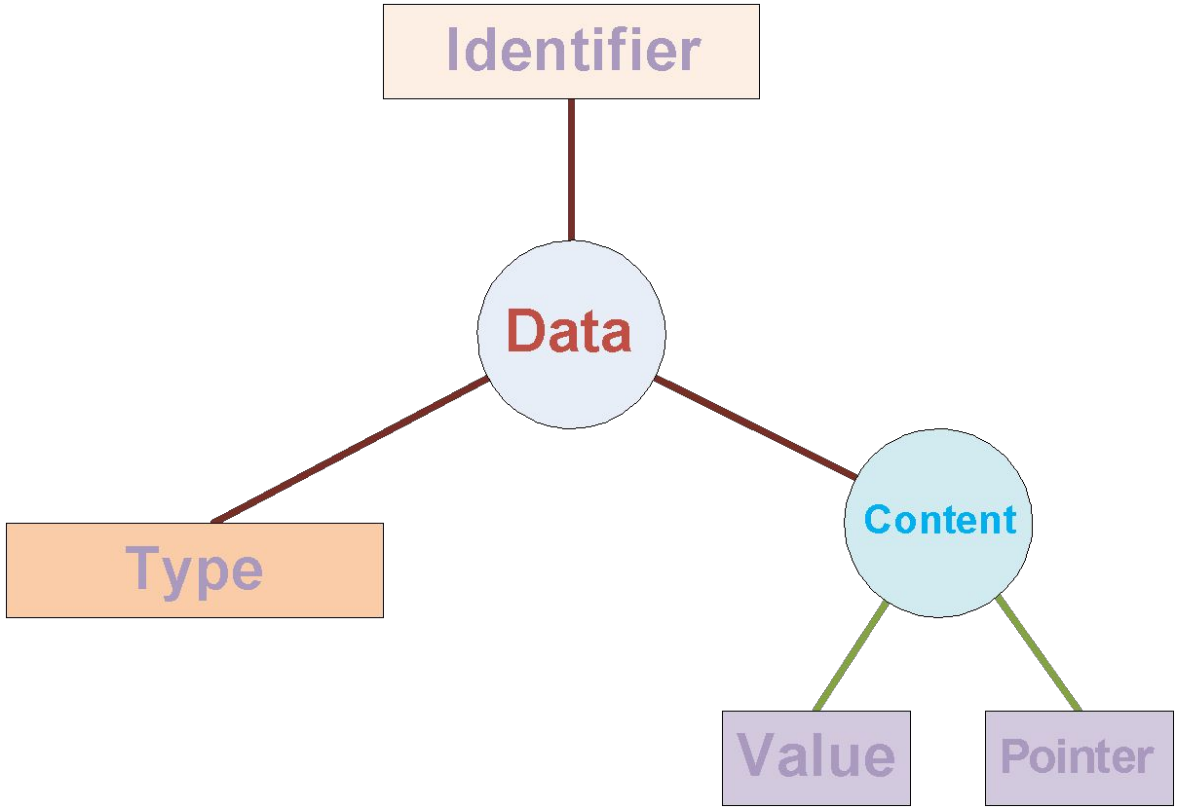
Concept of Pointer

- A pointer is a variable that represents the location (rather than the value) of a data item.

```
int x = 5;
```

Identifier	:	x
Value	:	5
Memory location	:	4576

Concept of Pointer



Concept of Pointer

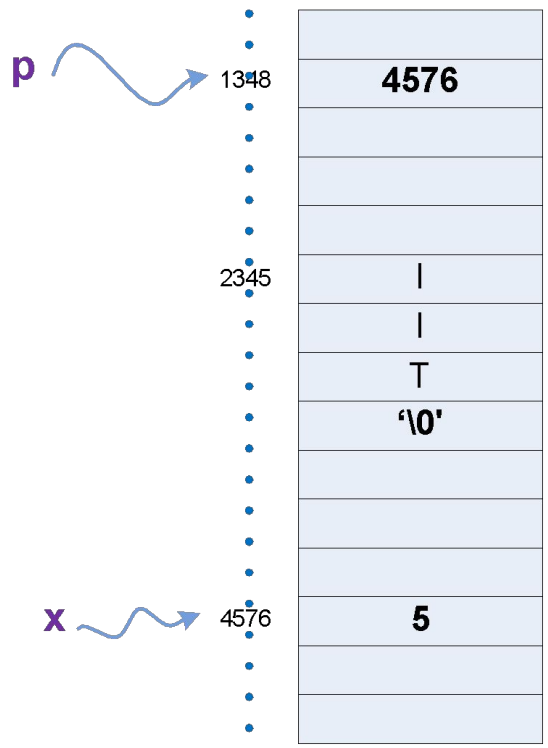
- In many programs, it is required to know the memory locations of some identifiers.
  - That is, we need to store memory addresses.

- Since memory addresses are simply numbers, they can be assigned to some variables which can be stored in memory.
  - Such variables that hold memory addresses are called pointers.
  - Since a pointer is a variable, its value is also stored in some memory location.
  - Such a variable is called pointer variable.

Concept of Pointer

- Suppose, we store the pointer of `x` in `p`

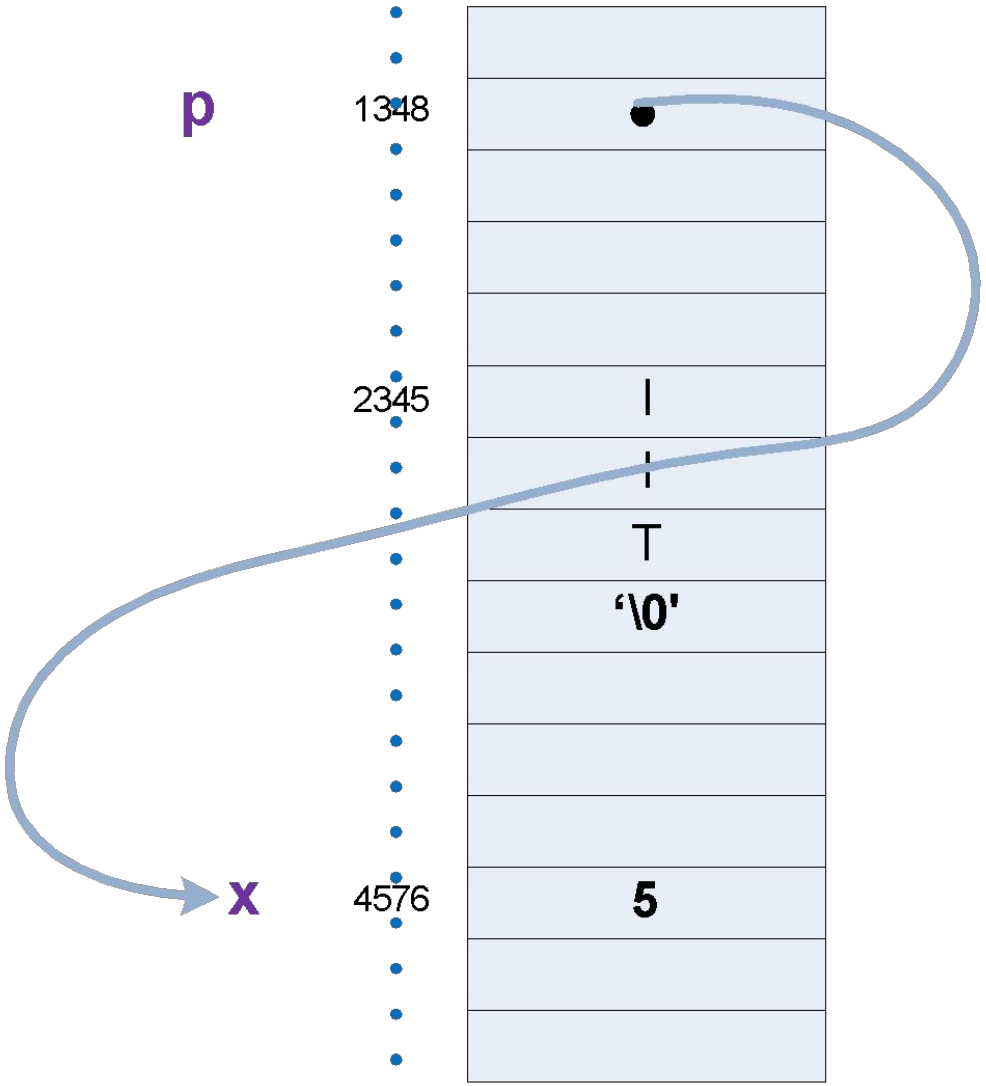
Variable	Address	Value
x	4576	5
p	1348	4576



In C, we write it as

```
p = &x;
```

Concept of Pointer



Applications of Pointer

- They have a number of useful applications.
  - Enables us to access a variable that is defined outside the function.
  - Can be used to pass information back and forth between a function and its reference point.
  - More efficient in handling data tables.
  - Reduces the length and complexity of a program.
  - Sometimes also increases the execution speed.

Pointer Manipulation in C

Declaration of a Pointer Variable

- Syntax in C

```
data_type  *pt-var_name;
```

- This tells the compiler three things about the variable `pt_var_name`
  - The asterisk * tells that the variable `pt_var_name` is a pointer variable
  - `pt_var_name` needs a memory location
  - `pt_var_name` points to a variable of type data_type
- Examples

```
int *x;          // A variable to store address of an integer variable
char *name;      // A variable to store address of a string variable
float *y;        // A variable to store address of a float variable
```

Initialization of a Pointer Variable

- Once a pointer variable `pt_var_name` is declared, it can be made to point to a variable `var_name` using an assignment statement such as

```
pt_var_name = & var_name        // Here, & is a unary operator
```

Example:

```
int x = 179, *p;
p = &x;
```

- A pointer variable can be initialized in its declaration itself.

Example:

```
int x, *p = &x;
```

Note:

```
int *p = &x,  x;
```

is invalid!

Pointer Variables

```
#include <stdio.h>

main(){
   int a;
   float b, c;
   double d;
   char ch;
   a = 10; b = 2.5;
   c = 12.36; d = 12345.66;
   ch= 'A';
   printf("a is stored in location  %u \n", &a);
printf("b is stored in location  %u \n", &b);
printf("c is stored in location  %u \n", &c);
printf("d is stored in location  %u \n", &d);
printf("ch is stored in location  %u \n", &ch) ;
}
```

Accessing a Variable through Pointer

- A pointer variable `pt_var_name` points a variable `var_name`. We can access the value stored in using the pointer.

- This is done by using another unary operator * (asterisk), usually known as the indirection operator.

```
value = * pt_var_name;
```

Example:

```
int x = 179, y, *p;
p = &x;
y = *p;
```

Note:

```
y = *(&x);
```

is a short-cut!

Pointer Arithmetic

- Like other variables, pointer variables can be used in arithmetic expressions. Suppose, `p1` is a pointer to `x` and `p2` is a pointer to `y`.

```
Examples:
  *p1 = *p2 + 10;          // Same as x = y + 10;

   y = *p2 + 1;          // Same as y = ++y;

  *p1 += 1;                // Same as x = x + 1;

  ++(*p1);                 // Same as *p1 = *p1+1;

  *p1 = *p2;               // Same as x = y;

   p1 = p2;                // p1 and p2 both points to y;
```

Pointer Arithmetic

- Following usage are illegal.

```
&235        // Pointing at constant is meaningless

int arr[20];
        . . .
&arr        // Pointing at array name is not done.
            // arr itself points to the starting location of the array

&(a+b)      // Pointing at expression is illegal

p = 1024;   // Assigning an absolute address to a pointer variable
```

Pointers and Arrays

Pointers and Arrays

- When an arrays is declared, the complier allocates a base address and sufficient amount of memory to store all the elements of the array in a contagious memory locations.

- The base address is the location of the first element of the array (index = 0).

- In C, there is a strong relationship between pointers and arrays.
  - Any operation that can be achieved by array subscripting can also be done with pointers
  - The pointer version, in general, is faster.

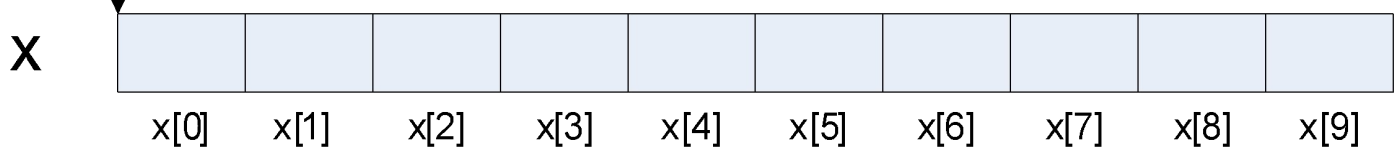
Pointers and Arrays

- Suppose, the is a declaration of an array of integers

```
int x[10];
```

- Name of the array, that is, x itself points to the starting location of the array. In other words

```
int *p;          // p is a pointer variable
p = &x[0];       // p points to the array
```



- `p` and `x` points to same memory location; however, `x` is not a pointer variable!
- `(p+i)` is the address of `x[i]`.

Pointers and Arrays

- If `p` points to an array `x`, then

  `p = x;` implies the current value of the pointer

  `*(p+i)` implies the content in `x[i]`

- Few things to be noted

  `x+i` is also identical to `p+i`

  `x[i]` can also be written as `*(x+i)`

  `&x[i]` and `x+i` are identical

  `p[i]` is identical to `*(p+i)`

  `p = x` and `p++` are legal *// Because, p is a pointer variable*

  `x = p` and `x++` are illegal *// name of an array is not a*

Pointers and Arrays

- When an array name is passed to a function, what is passed is the location of initial element.

- Within the called function, this argument is a local variable, and so an array name parameter is a pointer.

```
#include  <stdio.h>

int  arrayLen (char *s)
{
    int length;
    for (i=0; *s != '\0'; s++)
     length++;
    return (length);
}
```

Pointers and Arrays

- In the example, since `s` is a pointer, incrementing it is perfectly legal.

- `s++` has no effect on the content in the array, but just increments the private copy of the pointer.

- Following are all valid calls

```
#include  <stdio.h>

void  main()
{
    …..
    arrayLen(x);            // When, there exist say, char x[100];

    arrayLen("Debasis Samanta") ;        // Passed as a string

    arrayLen(ptr);                // When, there exist say, char *ptr;
}
```

Pointers and Arrays

- Passing an entire array to a function

```
#include <stdio.h>

float findMean (int x[], int size)
{
      int i, sum = 0;
      for (i=0; i<size; i++)
    sum += x[i];
      return ((float)sum/size);
}

void main()
{
      int a[100], i, n;

      scanf ("%d", &n);
      for (i=0; i<n; i++)
         scanf ("%d", &a[i]);

      printf ("\n Mean is %d",findMean( a,n);
}
```

Pointers and Arrays

- It is possible to pass a part of an array to a function.

Example:

Suppose, a function is defined as

```
foo(int x[]);
```
or
```
foo(int *x);
```

Then

```
foo(&a[i]);
```

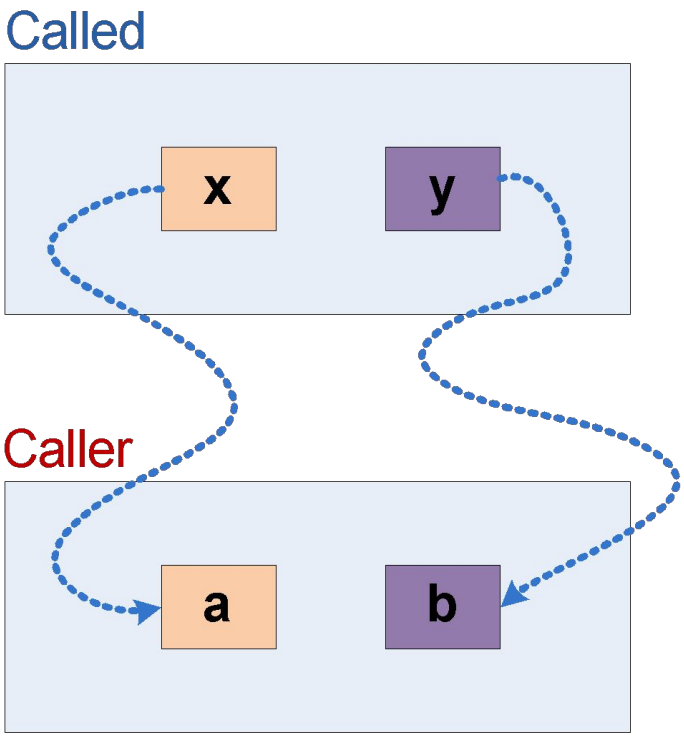
and

```
foo(a+i);
```

- Both pass to the function foo the address of the subarray that starts at a[i].

Pointers as Function Arguments

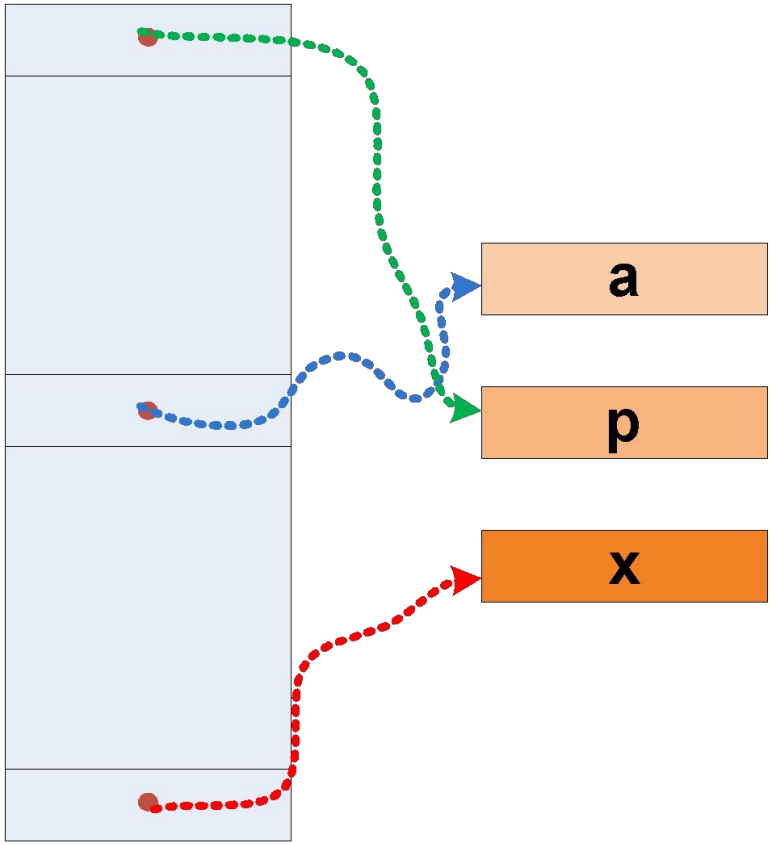
```
#include <stdio.h>

void swap (int *x, int *y)
{
      *x = *x + *y;
      *y = *x - *y;
      *x = *x - *y;

      return;
}

void main()
{
      int a, b;

      scanf ("a = %d, b = %d", &a, &b);
      swap(&a, &b);

      printf ("a = %d, b = %d", a, b);
}
```



Pointer Array

Pointer Array

- If an array stores pointers of some variables, then it is called a pointer array.




```
char * names[100];
```

// It stores pointers to 100 strings

```
int *marks[ ];
```

// It stores pointers to undefined `int` *values*

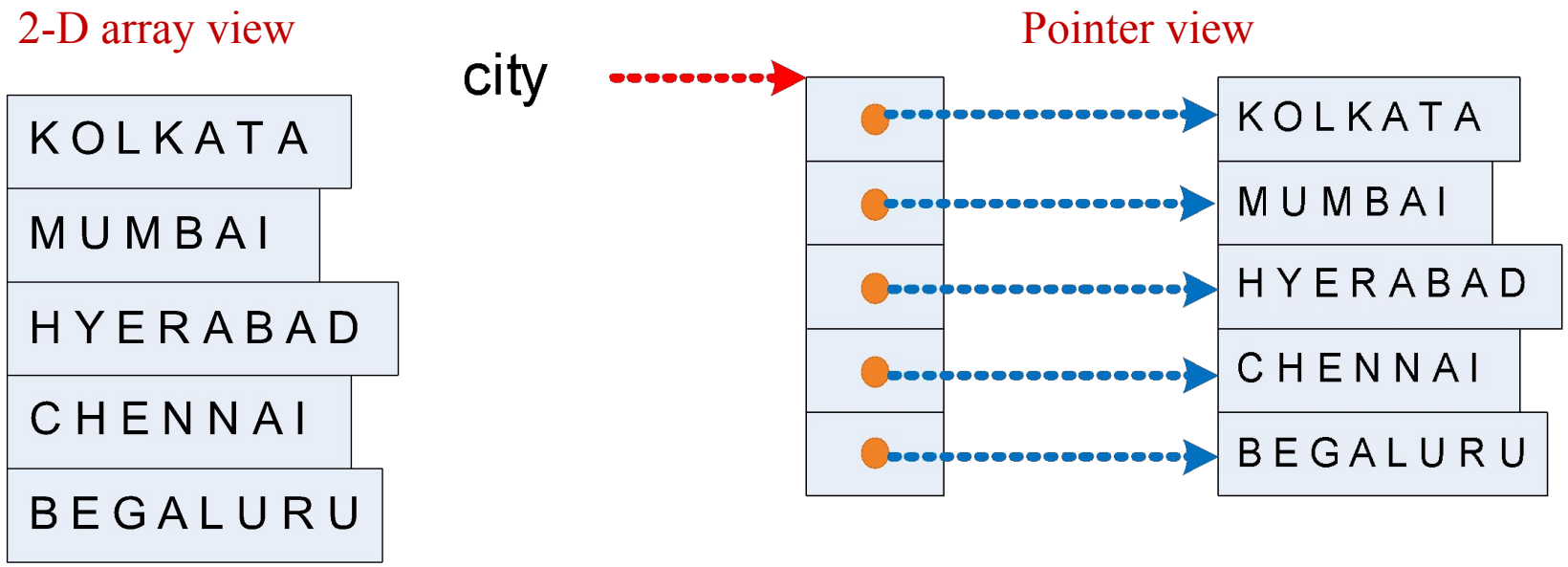
```
float **age;
```

// It stores pointers to undefined `float` *values*

Pointer and 2D Arrays

- We often use list of strings in many occasions
  - Names of cities, names of students, etc.

- Such a list of strings looks like a 2-D character array and better can be processed from the pointer view



Pointer and 2D Arrays

```
char city[20][15];        // 2-D array view: to store names of 20 cities
                          // of maximum 15 characters

char *city[20];           // Pointer view: to store names of 20 cities
                          // of maximum 15 characters

char *city[20] = {"Kolkata", "Mumbai", "Hyderabad",
"Chennai, Bengaluru"};    // Initialization of pointer array

printf("%s", city[i]);    // Using 2-D array view

printf("%s", city+i);     // Using pointer view
```

Command Line Arguments

Command Line Arguments

- It is a technique to supply input when the program is invoked

```
a.out Hello C!
    findMin 55 33 66 88 44 22 11 99 77
```

- This can be done by mentioning two arguments in `main( )`

```
void main(int argc, char *argv[]){
    ……
    state the body
     ……
    return;
  }
```

argc and argv

- **argc** : is an argument counter that counts the number of arguments on the command line

- **argv** : is an argument vector an represents an array of character pointers that point to the command line arguments

```
void main(int argc, char *argv[]){
   .........
   state the body
      .........
    return;
 }
```

For a.out Hello C!

argc = 3

argv[0] ------ a.out
argv[1] ------ Hello
argv[2] ------ C!

Command Line Arguments: Example

```
#include <stdio.h>

void main(int argc, char *argv[])
{
      int i;

    /* Print the values given as command line arguments */
      for(i=1;i< argc; i++)
          printf("%s%s",argv[i], (i<argc-1) ? " " : "");

      printf("\n");
      return;
}
```

Command Line Arguments

- Rename the `a.out` to a command name, for example, `findMin` as if the main function is renamed as `findMin`

- `argc = 1` implies that there is no command line argument

- All command line arguments, if any, are read as string and store them in the pointer array `argv`

- Declare `int *argv[]` as the second argument in the `main()` instead of `char *argv[]`, if you want to pass integer values and etc.

Pointers to Functions

Pointers to Functions

- A function, like a variable, has an address location in the memory.
- It is therefore, possible to declare a pointer to a function
  - Which then can be used as an argument in another function

- A pointer to a function is declared as follows

```
<type> (*fptr)();
```

//This is the simplest way of declaration

  - This tells the complier that `fptr` is a pointer to a function which returns `<type>` value

- Alternatively, a pointer to a function also can be declared as

```
<type> (*fptr)(<type1>, <type2>, . . .);
```

Or

```
<type> (*fptr)(<type1><agrg1, <type2><arg2>, . . .);
```

Pointers to Functions : Example

Example

```
double (*p)();              // p is a pointer to a function
double add(double x, double y);  // a function declaration
......
p = add;                    // pointer to function is assigned to p
    ... ...
 add(a,b);                  // The function add() is called with its arguments

(*p)(a,b);                  // This is equivalent to add(a,b)
```

Passing Functions to other Functions

- A pointer to a function (say, *guest*) can be passed to another function (say, *host*) as an argument.
  - This allows the guest function to be transferred to host, as though the guest function were a variable.
- Successive calls to the host function can pass different pointers (i.e., different guest functions) to the host.
- When a host function accepts the name of a guest function as an argument, the formal argument declaration must identify that argument as a pointer to the guest function.

Passing Functions to other Functions

- **Declaring a guest function**
  - Guest function can be declared as usual

```
<type_g> gfName(<type1>(arg1), <type2><arg2>, . . .);
```

- **Declaring a host function**
  - Host function can be declared as usual

Pointer to guest function

```
<type_h> hfName(..., <type_g>(*)( ),
    . . . data types of other arguments in
host);
```

- Note that the indirection operator '*' appears in parenthesis, to indicate a pointer to the guest function. Moreover, the data types of the guest function's arguments follow in a separate pair of parenthesis, to indicate that they are the function's arguments.

Passing Functions to other Functions

```
#include<stdio.h>

int process(int, int(*)(), char *);     // Declaration of a host function
int func1(int, int);                    // Declaration of a guest function
int func2(int, int);                    // Declaration of another guest function

void main(){
   int i, j; char name[20]);
   ...
    i = process(i, func1, name);
   ...
    j = process(j, func2, name);

    return;
}
```

Contd…

Passing Functions to other Functions

```
int process(int x, int (*pf)(), char *s) {
       int a, b, c;
       ...
       c = (*pf)(a, b);                    // Access the guest function
    ...
    return(c);
}

int func1(int a, int b) {
     int c;
      ...                    // Code in the guest function
      return(c);
}

int func2(int a, int b) {
     int d;
      ...                         // Another code in the guest function
      return(d);
}
```

Pointers to Functions : Example

```
int *readDataN();                          // Read the numbers to be sorted
char *readDataS();                         // Read the string data to be sorted
int compN(int x, int y);                   // To compare two numbers
int compS(char *x, char *y);               // To compare two strings
void swap (void *, void *);                // To swap two values
void sort(void *p, int (*q)(void *,void *),void (*r)(void *,void *));

void main(int argc, char *argv[])
{
      void *pInt, *pString;
      if (argc > 1) && strcmp(argv[1],"-n")== 0){
         pInt = readDataN(); q = compN; r = swapN;
         sort(pInt, q, r);}
      else {pString = readDataS(); q = compS; r = swapS;
         sort(pString, q, r); }
   return;
}
```

Problems to Ponder…

1. Is the following correct? If not, why?

```
float x;   int *p; p = &x;
```

2. What amount of memory would be require to store the values of the two pointer variables `p` and `q` as declared below?

```
float x, *p;    int y, *q;
```

3. Is the following statements valid? If not, why?

```
int *count;   count = 1024;
```

4. How one can print the a) value stored in a pointer variable and b) memory location, where a pointer variable is stored?

5. A pointer essentially stores a memory location. Now, amount of byte to store a memory location of a variable of any type is same. Does a pointer array can store pointers of any type of variables?

Problems to Ponder…

6. Consider the following declarations.

```
int *x, *p;
char a[20];
p = &x;   x = 100;
```

Give the answer to the following.

a) How to print the address of x:

b) What *p indicates?

c) How to print the address where p is stored?

d) How to print the content (i.e., address) which is stored in p?

e) How to access the variable through p?

f) can we write p = 0xabc6, where the right-hand side is a hexa-decimal number?

Problems to Ponder…

7. Given that

int *p, x, y;

Tell (in terms of equivalent C statement) the implication of the following.

```
a)  p = &x;        y = *p:

b)  y = *&x;

c) *p = 255;

d) p++;

e) y += *p;
```

Problems to Ponder…

8. Consider the following statement

```
int a[10];
int *p;
```

With respect to the above declaration, which of the following statements is/are illegal?

a) `p = a;`

b) `a = p;`

c) `a+i;`

d) `a++`

e) `P + i;`

Problems to Ponder…

9. Consider the following program

```
 include <stdio.h>
 main() {
    int i = 3, *j, **k;
    j = &i;      k = &j;
    printf("a is %u", &i);
    printf("b is %u", j);
    printf("c is %u", k);
printf("d is %u", &j);
printf("e is %u", &k);
printf("f is %d", i);
    printf("g is %d", *j);
printf("h is %d", *(&i));
printf("i is %d", **k);
}
```

What the output a, b, c, …, I signify? Draw an appropriate memory instance and the explain your answers.

Problems to Ponder…

10. Using command line arguments say `void main(in atrgc, char *argv[])` we pass string values as input to the program. What modification should be in the argument list, if any, if we have to pass the input other than string values?

11. What is the difference between the two declarations?

```
<type> (*f());
<type> *f();
```

If you try to solve problems yourself, then you will learn many things automatically.

Spend few minutes and then enjoy the study.